CS5013 Final Project Report

IITM Explainable Slotbook

Nishanth S, EE23B049 Sparsh Gupta, EE23B117

Team Name: [TODO: team name]
Repository: `https://github.com/lazarsalt1441/CS5013-PROJECT`
Submission commit: `[TODO: SHA of the final-submission tag]` (date: `[TODO: YYYY-MM-DD]`).
Docker image: [TODO: Google Drive link to slotbook-¡sha¿.tar.gz]
Tagline: *It builds an IITM department's weekly lecture-slot timetable from course and preference data without breaking any hard rule, and explains why each course got its slot.*

AI-authorship clause. *We used AI tools to help draft parts of this report. The claims, numbers, commit references, stakeholder statements, and the Docker image are our own; the prose shaping may have been assisted. We can defend every sentence of this report, and every line of code in the submission commit, in the viva.*

1 Executive summary

Departments at IITM assign lecture slots by hand-editing last year's slotbook. Nobody can then say why a course got its slot, or what a clash would cost to fix.

We shipped a Java engine that validates five CSV inputs, solves with UniTime's cpsolver (an exact CP-SAT model is kept as a reference), and writes the slotbook with a reason for every assignment and every rejected slot. Students and professors enter preferences on a Vercel portal backed by Postgres, which also warns about missing prerequisites using the student's grade card. The timetabler downloads the CSVs from the admin page, runs one command, and publishes the result. The UniTime solver, the portal and prerequisite warnings were added after the proposal. [TODO: check this sentence against your proposal]

Stakeholder verdict: Accepted (final demo on [TODO: date]).

2 Course concepts used in the project

Java paths are under `src/main/java/in/ac/iitm/slotbook/`.

- **Module 2 – Prompt engineering (CoSTAR, negative prompts).** All 19 AI tasks were logged before coding, as the request, a CoSTAR restatement and negative constraints added at review. `prompts/PROMPT_LOG.md`; `ebb9c28` (P015, logged before `7591cca`).
- **Module 3 – Git, Conventional Commits and CI.** Branches and pull requests; GitHub Actions runs both suites against a Postgres service, plus CLI runs on both solvers. `.github/workflows/ci.yml`; `8f2c000` (PR #1).
- **Module 4 – Structured debugging.** 14 faults recorded from reproduction to regression test, e.g. a race that returned 500s under 25 concurrent writers. `docs_v3/DEBUGGING.md`; `2607a9d`.
- **Module 4 – Static analyser first, then the smallest patch.** SpotBugs reported 10 findings; 3 were a real bug (day names broke under a Turkish locale). Now 0, and any new one fails CI. `spotbugs-exclude.xml`; `374d4e7`.
- **Module 5 – Independent oracles and stress testing.** A separate checker, penalty calculator and brute-force solver judge both solvers; fuzzing and 300 concurrent requests found 3 faults. `src/test/.../testing/ConstraintChecker.java`; `efdf10d`, `2607a9d`.
- **Module 5 – Property-based and mutation testing.** jqwik found on its first run that meetings that only touch were counted as clashes; PIT raised the mutation score from 91.0% to 98.7% (2 equivalent mutants left), now gated at 97%. `src/test/.../model/SlotPropertiesTest.java`; `82cc7f1`, `fc462ab`.

- **Module 5 – Documentation as code.** A `slotbook(1)` man page linted by CI, a manual, and a test plan listing every test and tool result. `man/slotbook.1`, `docs_v3/TEST_PLAN.md`; `9ee919b`.
- **Module 7 – Licences and provenance.** Licences of all 16 runtime dependencies are generated in CI and shipped in the image; LGPL cpsolver stays a replaceable jar. `THIRD_PARTY_NOTICES.md`; `406b360`.
- **Module 9 – Supply chain and secrets.** Lockfile with `npm ci`, `npm audit` gate (0 vulnerabilities), `DATABASE_URL` only from the environment, a non-root image scanned for secrets. `.github/workflows/quality.yml`; `95fb07f`, `f47ae1c`.

3 AI-usage summary

Tools used. Claude Code **[TODO: %]** (engine, portal, tests, docs); ChatGPT **[TODO: %]** (Vercel and Neon deployment). **[TODO: other tools?]**

One substantive help. Claude Code replaced the Google Sheets backend with a Postgres store, numbered migrations and database-backed tests (`portal/api/_store.js`, `7591cca`); it shipped near as-is once 4 integration tests passed on a real Postgres 16.

One substantive failure caught in review. The AI-written admin-dashboard check counted U and W grades as completed, contradicting our rule (a specification-vs-implementation gap, Module 5); a golden file shared with the Java CLI caught it before merge (`portal/api/_prerequisites.js`, `f528505`).

Net-effect sentence. **[TODO: your honest verdict]**. For reference, `docs_v3/DEBUGGING.md` records 14 faults in AI-assisted code, every one caught by a test or a tool rather than by reading the code.

4 Deployment and stakeholder handoff

Build and run.

```
docker load -i slotbook-<sha>.tar.gz
docker run --rm slotbook:<sha>        # solves data/feasible, prints the slotbook
mkdir -p out && docker run --rm -u "$(id -u):$(id -g)" -v "$PWD/out:/out" \
    slotbook:<sha> --input data/demo-showcase --output /out   # CSVs land in ./out
```

[TODO: replace ¡sha¿ with the 7-character SHA of the final-submission commit] The image holds the engine and CLI. The web portal runs on Vercel and is not part of it.

Where the artefact lives after the course. Code on GitHub (above); portal at `https://cs5013-project-portal.vercel.app`. **[TODO: who keeps it running after the course?]**

Stakeholder quote (one sentence, verbatim).

> *"[TODO: stakeholder's exact words from the final demo email]"*
> *– [TODO: Name, role], [TODO: date].*

5 Known limitations

- **Optimality.** The default UniTime solver is a heuristic: on 3 of the 13 generated scenarios with a proven optimum it finished 1–8 points above CP-SAT. `--solver cpsat` is needed for a guarantee.
- **No identity on the portal.** Anyone can submit under any student ID, and grade cards are self-uploaded and unverified, so warnings are only as trustworthy as the uploader. It is not fit for real student data until IITM login exists.
- **Fixed rooms, one PDF layout.** Rooms are fixed per course; capacity is not modelled. The grade-card parser reads only the IITM PDF's text layout, so scanned or photographed cards are rejected.

6 Individual contributions

- **Nishanth S:** v1 engine (CP-SAT model, explanations, CLI, JavaFX view, CI); the Vercel preference portal on Google Sheets; the mid-evaluation showcase dataset and submission pack. **[TODO: confirm; add testing and stakeholder liaison]**
- **Sparsh Gupta:** testing framework (independent oracles, generated scenarios, brute force); v2 UniTime wrapper and real IITM time grid; the prompt log; v3 Postgres backend, grade-card prerequisite warnings, stress tests, property and mutation testing, man page and docs. **[TODO: confirm; add stakeholder liaison]**